

Yes—ARCHER could reasonably coordinate specialized models. I recommend one consistent assistant that delegates to a small number of specialist workers, with reliable task tracking underneath. You should experience one ARCHER, without having to manage its internal team.

The biggest opportunity for you is **reliable follow-through**: remembering commitments, preparing the next step, completing administrative work, and preserving context when you return.

I retrieved relevant findings from your Behavioral Operating Profile discussion. They distinguish your strengths in research, synthesis, systems design, and meaningful creative work from the disproportionate burden of fragmented logistics and follow-up chains. That distinction should drive the design.

1. Separate specialist roles from separate models

A specialist can use the same underlying LLM with different instructions, tools, and project context. Giving a model a “business expert” persona does not itself create expertise; it needs reliable sources, appropriate tools, and measurable results.

Architecture	What it means	Fit for ARCHER
One model, several specialist workflows	Different instructions and tools for research, planning, coding, etc.	Simplest starting point
Hybrid model routing	Everyday work uses an economical model; demanding work goes to a stronger specialist	My recommendation
Several local models	Models load or run on your computer as needed	Useful for privacy; constrained by memory and competing workloads
Several cooperating agents	Workers independently investigate or execute subtasks	Useful selectively, especially for research
Custom-trained specialists	Fine-tune models on narrowly defined tasks	Later, after collecting evidence that simpler approaches fail

More agents can add coordination errors, latency, and expense. Anthropic’s research system found benefits on parallel research tasks, but also reported substantially greater

token consumption. That is evidence for selective delegation—not making every conversation a committee. Multi-agent research findings

2. The specialist roles I would give ARCHER

Your current repository already has a central `CoreAgent` with cloud-delegation logic. Your earlier planning documents also describe Builder, QC, and DevSteward responsibilities. This can be an extension of that foundation.

Role	Responsibilities	What makes it useful
Executive assistant	Conversation, priorities, commitments, session continuity	Understands your preferences and maintains a coherent plan
Research and technology scout	Investigates tools, models, competitors, and specific capability gaps	Produces sourced comparisons and testable recommendations
Builder	Creates websites, integrations, automations, and fixes	Works in isolated branches and produces running previews
Verifier	Checks requirements, tests behavior, reviews evidence	Can reject a claimed success when the result doesn't work
Business operator	Tracks validation, acquisition, revenue, support, and experiments	Connects development work to customer outcomes

These are **five responsibilities, not necessarily five models**. Initially, I'd use one general model for several roles and a stronger coding/reasoning model for difficult work. Use a different model for review when its independent perspective proves valuable.

Schedulers, calculations, reminders, and database updates should largely be ordinary software. An LLM should not have to “remember” to wake up tomorrow.

3. Design the assistant around your actual operating patterns

The goal is to reduce the work required to stay organized without interrupting the substantive work you do well.

Pattern from your profile	Proposed ARCHER behavior
Follow-up chains become burdensome	Track the dependency: “When this arrives, check it, update the record, and prepare the next action.”
Returning to a task requires rebuilding context	Maintain a short re-entry note: where you stopped, what changed, and the exact next action
Exploration leads to later refinements	Distinguish exploring , decided , and approved to build ; preserve revisions without treating every idea as a new assignment
Too many open loops compete for attention	One capture inbox; a proposed limit of two active projects; everything else stays in a visible parking lot
Vague tasks create initiation friction	Prepare a concrete first action, ideally executable in 10–15 minutes
Long responses can become hard to process	Lead with the decision and next action; keep supporting detail available
Novelty can pull attention away from completion	Capture the new idea, then show what switching would delay—without scolding

For example, instead of:

“Remember to work on your website.”

ARCHER should say:

“The checkout test passed. The remaining blocker is choosing the launch price. I’ve prepared two options; that’s the only decision I need from you.”

Proactivity should primarily mean doing useful preparation in the background.

Interrupt only when there is a time-sensitive issue, a decision blocking progress, or something you specifically asked it to watch. Repeatedly dismissed suggestions should become less frequent.

I would also avoid treating camera expressions, silence, or inactivity as reliable evidence of your emotional state. Your own statements and explicit preferences should carry more weight.

4. Give it durable memory and durable work

This matters more than adding another model.

ARCHER needs distinct records for:

- **Personal preferences:** communication style, boundaries, and working accommodations.
- **Project truth:** current requirements, approved designs, decisions, and source files.
- **Execution state:** what is running, blocked, completed, or waiting on someone.
- **Evidence:** test results, screenshots, source links, and actual outputs.

Every task should have an owner, next action, completion criteria, last checkpoint, and supporting evidence. It should survive a restart and resume without duplicating external actions.

Your existing stack could implement this incrementally. LangGraph is another option if checkpointing and resumable workflows become difficult to maintain yourself; it supports persistent execution state and longer-term stores. Adopting it would be an engineering choice, not a prerequisite. Persistence documentation

Keep your existing local-only treatment of sensitive profile information. Cloud specialists usually need a focused task brief, not your entire personal history.

5. Make technology scouting answer real problems

I would give the scout a prioritized list of capability gaps—for example, speech cost, transcription accuracy, browser reliability, or memory retrieval—and a bounded research budget.

Its proposed workflow:

1. Identify a measurable weakness in ARCHER.
2. Find credible candidate improvements.
3. Check compatibility, licensing, maintenance, and total operating cost.
4. Test promising candidates against the current implementation.
5. Present an evidence-backed recommendation.
6. Prepare an isolated change with a rollback path.

A useful recommendation would be:

“This provider reduced speech cost in our test, but added response delay. Here are the voice samples and measured results.”

That is more valuable than a daily stream of new AI releases.

Self-improvement should mean a controlled engineering process. ARCHER can research, prototype, and test within standing authorization. It should not independently

broaden its permissions, expose private data, or replace its live infrastructure merely because a new tool looks promising.

6. Make business building a complete operating workflow

ARCHER should help you move through:

Customer problem → evidence of demand → smallest sellable offer → acquisition test → delivery → retention.

Building the website is only one part. A technically polished product can still fail because nobody wants it, cannot find it, or costs too much to support.

For each business, ARCHER should maintain:

Question	Required evidence
Who has the problem?	Specific customer group and observed pain
Will they pay?	Customer conversations, paid tests, or other credible demand signals
How will they find it?	A concrete acquisition channel and experiment
Is it worth operating?	Revenue, service costs, support burden, and your time
Should we continue?	Predetermined continue/change/stop criteria

For you, I would favor **one business experiment at a time**, with ARCHER handling the associated tracking and preparation. It should surface the smallest decision you need to make, then carry out the authorized work around it.

There is a particular risk here: **improving ARCHER can become an absorbing project that continually postpones using ARCHER to generate income.** I'd assign assistant-development work a fixed capacity budget and require most improvements to remove a current obstacle in your life or business.

My proposed implementation order

1. **Reliable continuity:** commitments, task state, re-entry notes, and evidence-based completion.
2. **Builder and verifier:** one bounded task through implementation, testing, and review.
3. **Model routing:** compare candidate models on representative ARCHER tasks; add specialists only where they improve results.

4. **Bounded technology scouting:** recommendations tied to measured deficiencies.

5. **Business execution:** take one offer through validation and a small launch.

Measure success by **completed commitments, fewer repeated instructions, faster task re-entry, fewer unwanted interruptions, cost per successful task, and business results.**

I would start with the executive-assistant workflow and the builder/verifier pair. Those directly address your follow-through burden while giving ARCHER a practical way to improve itself and build useful products.